International Institute of Information Technology, Hyderabad

CL3.410 - Language Models and Agents: Major Project Report

Periodic Table SME Agent

An Autonomous Retrieval-Augmented Reasoning System

Team Aurors

Vivek Hruday Kavuri 2022114012
Karanam Vysishtya 2022102044

Project Specifications

Core Architecture: *LangGraph Orchestration + RAG*

Retrieval Stack: *Elasticsearch (Hybrid) & ChromaDB*

Key Capabilities: *Automated Quiz/Report Generation, PDF/PPTX Creation, Email Automation*

Course Instructor
Prof. Vasudeva Varma

Monsoon 2025

Contents

1 Data Preparation and Ingestion Pipeline **3**
1.1 Document Collection and Format Support . . . 3
1.1.1 OCR Fallback Capability [Bonus] . . . 3
1.2 Preprocessing and Data Cleaning . . . 3
1.3 Chunking Strategy . . . 4
1.3.1 Strategy Justification . . . 4
1.3.2 Hierarchical Levels . . . 4
1.4 System Architecture and Performance Features . . . 4
1.4.1 Batch Ingestion via Multiprocessing [Bonus] . . . 4
1.4.2 Logging and Error Handling . . . 4
1.5 Output Artifacts . . . 4

2 Embedding and Indexing Pipeline **5**
2.1 Embedding Generation Strategy . . . 5
2.1.1 Model Loading and Configuration . . . 5
2.1.2 Manual Pooling and Normalization . . . 5
2.2 Vector Database Implementation: Elasticsearch . . . 5
2.2.1 Index Mapping and Schema . . . 5
2.2.2 Batch Indexing Process . . . 6
2.3 Execution Modes . . . 6

3 Retrieval and Relevance Ranking Strategy **6**
3.1 Hybrid Retrieval Mechanism . . . 6
3.2 Relevance Reranking . . . 6
3.3 "Small-to-Big" Retrieval Strategy . . . 7
3.4 Dynamic Resource Management . . . 7

4 RAG - Generation Pipeline **7**
4.1 Generative Model Integration . . . 7
4.2 Prompt Engineering and Context Construction . . . 8
4.2.1 Context Formatting . . . 8
4.2.2 System Prompts . . . 8
4.3 Pipeline Workflow . . . 8
4.4 Error Handling . . . 8

5 Agentic Workflow and Orchestration **8**
5.1 State Management . . . 9
5.2 Intent Extraction and Planning . . . 9
5.3 The Dynamic Task Loop . . . 9
5.4 Self-Correction and Dependency Injection . . . 10

6 Tooling and Functional Capabilities **10**
6.1 Knowledge Retrieval Tool . . . 10
6.2 Content Generation Tools . . . 10
6.3 Document Generation Capabilities . . . 10
6.3.1 Professional PDF Generation . . . 10
6.3.2 Presentation Generation (PPTX) . . . 11
6.4 Communication: Smart Email Tool . . . 11

7 System Architecture and Implementation **11**
7.1 Backend API Server . . . 11
7.1.1 Server Configuration . . . 11
7.1.2 Streaming Execution Endpoint . . . 11
7.2 Data Persistence and Memory Management . . . 12
7.2.1 Database Schema . . . 12
7.2.2 Intelligent Summarization Module . . . 12
7.3 Frontend Interface . . . 12

7.3.1 User Experience Features . . . 12
7.3.2 Session Management . . . 12

8 Experiments and Results **12**
8.1 Quantitative Analysis . . . 13
8.2 Qualitative Analysis . . . 13
8.2.1 Reasoning Capabilities: Flash vs. Lite . . . 13
8.2.2 Consistency and Instruction Following . . . 13
8.2.3 Viability of Local Models . . . 13

9 Bonus Implementations **13**
9.1 Automated Batch Ingestion Pipeline . . . 14
9.2 Relevance Reranking Mechanism . . . 14
9.3 Self-Learning and Adaptation via Feedback Loops . . . 14
9.4 OCR Fallback for Scanned Documents . . . 14
9.5 Input Guardrails and Sanitization . . . 14

1 Data Preparation and Ingestion Pipeline

The foundation of the Subject Matter Expert (SME) Agent is a robust data ingestion pipeline designed to handle heterogeneous document formats, ensure data quality through rigorous preprocessing, and implement a hierarchical chunking strategy optimized for Retrieval-Augmented Generation (RAG). The implementation is encapsulated within `ingestion.py`.

1.1 Document Collection and Format Support

The system is designed to ingest data from a root-level directory, recursively identifying valid source documents. To ensure the SME can handle diverse knowledge bases, the pipeline supports the following formats via specific Python libraries:

- **PDF Documents (.pdf):** Processed using `pdfplumber`. The system iterates through pages to extract text while maintaining layout awareness.
- **Word Documents (.docx):** Processed using `python-docx`, extracting text paragraph by paragraph.
- **Presentations (.pptx):** Processed using `python-pptx`, aggregating text from slides and shapes.
- **Structured Data (.csv):** Processed using the standard `csv` library with `csv.Sniffer` to automatically detect dialects and delimiters.
- **Plain Text and Code (.txt, .md, .py, .json, etc.):** Ingested via standard UTF-8 or Latin-1 decoding.

1.1.1 OCR Fallback Capability [Bonus]

To address the challenge of scanned or image-based PDF documents where standard extraction fails, the pipeline implements an Optical Character Recognition (OCR) fallback mechanism.

- **Trigger:** If standard extraction yields minimal text (less than 20 characters) for a page, the system triggers the OCR module.
- **Implementation:** The system utilizes `pytesseract` (Tesseract-OCR engine) combined with `Pillow` (PIL) to render the PDF page as an image at 200 DPI and extract text from the visual layer.
- **Logging:** The system logs instances where OCR was utilized to distinguish between natively digital and scanned content.

1.2 Preprocessing and Data Cleaning

Raw text extracted from documents often contains noise that degrades embedding quality. The `clean_text` function implements a multi-stage normalization pipeline:

1. **Heuristic Filtering:** The `remove_non_informative` function filters out headers, footers, and metadata artifacts using Regular Expressions (Regex).
   - *Pattern Matching:* Specific patterns are used to remove standalone dates (e.g., "11/02/2025"), timestamps, and page number indicators (e.g., "Page 1 of 12").
   - *Noise Removal:* Lines consisting predominantly of punctuation or non-alphanumeric characters are discarded.
2. **Whitespace Normalization:** All vertical whitespace (carriage returns) is converted to newlines. Multiple spaces are collapsed into single spaces, and excessive newlines (more than two) are reduced to a standard paragraph separator ($\backslash n \backslash n$).
3. **OCR Artifact Correction:**
   - *De-stuttering:* A regex substitution `r'(.)\1{2,}'` is applied to fix common OCR errors where characters are repeated unnecessarily (e.g., converting "nnnnnoooommmm" to "nom").
   - *Word Art Removal:* Spaced-out headings (e.g., "n o m e n c l a t u r e") are detected and normalized.
4. **Deduplication:** To prevent redundant information from polluting the vector index, the system computes a SHA256 hash of the cleaned text for every document. Duplicate hashes result in the document being flagged and skipped.

1.3 Chunking Strategy

The project implements a **Hierarchical Chunking Strategy** to facilitate both broad context retrieval and precise detail extraction. This approach addresses the "Lost in the Middle" phenomenon by creating parent-child relationships between text segments.

1.3.1 Strategy Justification

We utilize a **Content-Aware Paragraph Packing** strategy as the primary method, with a sliding window fallback.

- **Paragraph Packing (Default):** The algorithm treats paragraphs (delimited by double newlines) as atomic units. It attempts to pack complete paragraphs into a chunk until the token limit is reached. This preserves semantic integrity, ensuring that sentences or logical thoughts are not split mid-way, which is critical for maintaining the coherence required for the LLM to answer complex queries.
- **Sliding Window (Fallback):** For texts lacking clear paragraph structure, a fixed-size sliding window approach is used.

1.3.2 Hierarchical Levels

The pipeline generates chunks at three distinct levels of granularity, establishing a parent-child linkage:

- **Level 1 (Parent - 2048 Tokens):** Captures high-level context and broad thematic elements.
- **Level 2 (Child - 512 Tokens):** Provides standard context suitable for most queries.
- **Level 3 (Leaf - 128 Tokens):** Captures granular details and specific facts.

Each chunk object contains metadata linking it to its source document (`parent_id`) and, where applicable, its immediate parent chunk (`parent_chunk_id`). The standard overlap fraction is set to 0.1 (10%) to ensure semantic continuity across boundaries.

1.4 System Architecture and Performance Features

1.4.1 Batch Ingestion via Multiprocessing [Bonus]

To handle large corpora efficiently, the ingestion pipeline utilizes Python's `multiprocessing` library.

- The system detects the number of available CPU cores and spawns a corresponding number of worker processes.
- A `multiprocessing.Pool` manages the workload, distributing file processing tasks via `starmap`.
- This parallelization significantly reduces the time required for the text extraction phase, which is typically I/O bound and CPU intensive during OCR operations.

1.4.2 Logging and Error Handling

A dual-channel logging system is implemented:

- **Ingest Log (ingest.log):** Records successful extractions, token counts, and processing times.
- **Error Log (errors.log):** Captures stack traces for failed extractions (e.g., corrupted files, encoding errors) without halting the entire pipeline.

1.5 Output Artifacts

The pipeline produces two primary JSONL (JSON Lines) artifacts:

1. `docs_manifest.jsonl`: Contains metadata for every processed document, including file path, SHA256 hash, and extraction status.
2. `chunks.jsonl`: Contains the actual text chunks with their embeddings-ready metadata (Chunk ID, Parent ID, text content, token count).

2 Embedding and Indexing Pipeline

Following data ingestion, the system transitions to the Embedding and Indexing phase. This critical stage transforms the hierarchical text chunks into dense vector representations and stores them in a high-performance vector database. The implementation, defined in `index_and_embed.py`, utilizes a manual embedding generation pipeline using Hugging Face `transformers` and indexes data into `Elasticsearch`.

2.1 Embedding Generation Strategy

Unlike standard implementations that rely on high-level abstraction libraries, our system implements a manual embedding pipeline to ensure precise control over tokenization, pooling, and normalization. This allows the system to switch agnostic of the underlying model architecture (e.g., BGE, Gemma).

2.1.1 Model Loading and Configuration

The system dynamically loads models using the `AutoTokenizer` and `AutoModel` classes from the `transformers` library.

- **Device Acceleration:** The script automatically detects and utilizes hardware acceleration, prioritizing NVIDIA GPUs (`cuda`), followed by Apple Silicon (`mps`), and falling back to CPU if necessary.
- **Dimensionality Detection:** The system explicitly retrieves the hidden size configuration (e.g., 768 for base models, 2048 for Gemma-2b) to configure the vector database schema dynamically.

2.1.2 Manual Pooling and Normalization

To generate a single vector representation from the sequence of token embeddings output by the LLM, we implement a custom `mean_pooling` strategy rather than relying on the default pooler. The process involves:

1. **Tokenization:** Input text is tokenized with padding and truncation up to the specified `max_length` (e.g., 512 or 2048 tokens).
2. **Inference:** The model outputs the `last_hidden_state`, a tensor of shape $(Batch, SequenceLength, HiddenSize)$.
3. **Attention Masking:** To avoid incorporating padding tokens into the sentence embedding, the system expands the attention mask to match the embedding dimensions.
4. **Mean Pooling Operation:**
$$E_{sentence} = \frac{\sum(E_{tokens} \times Mask)}{\sum Mask} \tag{1}$$
This calculates the average of all valid token embeddings.
5. **L2 Normalization:** The resulting vectors are normalized using `torch.nn.functional.normalize` ($p = 2$). This step is crucial because it ensures that the dot product between two vectors is equivalent to their Cosine Similarity.

2.2 Vector Database Implementation: Elasticsearch

The project utilizes `Elasticsearch` as the vector store to support Hybrid Search (Dense Vector + Sparse Keyword).

2.2.1 Index Mapping and Schema

The system defines a strict mapping for the index to support efficient retrieval:

- **Dense Vector Field (`embedding`):** Configured with type `dense_vector` and `similarity: "cosine"`. The dimensions are set dynamically based on the loaded model.
- **Text Field (`chunk_text`):** Mapped as standard `text`. This enables the use of the BM25 algorithm for keyword-based retrieval, a requirement for our Hybrid RAG approach.
- **Metadata Fields:** Fields such as `parent_id`, `parent_chunk_id`, `source_path`, and `filename` are mapped as `keyword` to allow for precise filtering (e.g., "retrieve only chunks from this document").

2.2.2 Batch Indexing Process

To handle large datasets efficiently, the indexing process is batched:

1. The system reads the processed `chunks.jsonl` file.
2. It accumulates chunks into batches (configurable via `--batch-size`).
3. The batch is passed to the GPU for embedding generation.
4. The resulting vectors and metadata are packaged into actions and sent to Elasticsearch using the `bulk` helper API, which minimizes network overhead.

2.3 Execution Modes

The script supports distinct execution modes via Command Line Interface (CLI) arguments:

- `--mode index`: Initiatives the full indexing pipeline. It includes a safety flag `--delete-existing` to wipe and recreate the index schema during development iterations.
- **Configuration:** Users can specify the target `--model` (e.g., `BAAI/bge-base-en-v1.5`) and `--max-length` to adapt the pipeline for different SME domains without code changes.

3 Retrieval and Relevance Ranking Strategy

The retrieval engine is the core component that connects user queries to the ingested knowledge base. Implemented in the `Retriever` class within `retrieval.py`, our system employs a multi-stage pipeline combining hybrid search, semantic reranking, and a "Small-to-Big" context expansion strategy.

3.1 Hybrid Retrieval Mechanism

To maximize recall, the system queries the Elasticsearch index using a hybrid approach that balances semantic understanding with precise keyword matching.

- **Vector Search (k-NN):** The system computes the cosine similarity between the query embedding and stored chunk embeddings.
- **Keyword Search (BM25):** Simultaneously, a standard text match query is executed against the `chunk_text` field.
- **Implementation:** These two signals are combined within a single Elasticsearch query body:

```
{
    "knn": { "field": "embedding", "k": top_k, ... },
    "query": { "match": { "chunk_text": { "query": query, "boost": 0.1 } } }
}
```

  The explicit boost factor ensures that while vector similarity drives the primary retrieval, exact keyword matches (critical for specific terminology) refine the scoring.

3.2 Relevance Reranking

Raw similarity scores from vector databases often lack the nuance required to distinguish between "related" and "answer-containing" chunks. We implement a second-stage reranking process:

1. **Retrieval:** The top K (e.g., 20) candidates are fetched from Elasticsearch.
2. **Cross-Encoding:** A `CrossEncoder` model (e.g., BAAI/bge-reranker-base) takes pairs of $(Query, Document)$ and outputs a single relevance score. Unlike bi-encoders, the cross-encoder attends to the query and document simultaneously, capturing deep semantic interactions.
3. **Selection:** The candidates are re-sorted based on these new scores, and the top N (e.g., 5) are selected for context generation.

3.3 "Small-to-Big" Retrieval Strategy

A key innovation in this pipeline is the implementation of "Small-to-Big" (or Parent Document) retrieval.

- **The Problem:** Small chunks (e.g., 128 tokens) are excellent for vector matching because they contain specific, dense information. However, they often lack sufficient context for the LLM to generate a comprehensive answer.
- **The Solution:** The retrieval logic checks if a retrieved hit is a "child" chunk (indicated by the presence of a `parent_chunk_id`).
- **Expansion:** Instead of returning the small child chunk, the system automatically fetches and returns the **parent chunk** (e.g., 512 or 2048 tokens). This ensures the LLM receives the specific detail found by the vector search *plus* the surrounding context required for reasoning.
- **Deduplication:** The logic tracks `fetched_parent_ids` to ensure that multiple child hits from the same section do not result in duplicate context blocks.

3.4 Dynamic Resource Management

To run efficiently on limited hardware (e.g., consumer GPUs or cloud instances with limited VRAM), the `Retriever` class implements aggressive resource management:

- **Just-in-Time Loading:** Models (Embedding and Reranker) are stored in system RAM (CPU) by default.
- **Context Switching:** When a search is requested:
  1. The embedding model is moved to the GPU/MPS device to encode the query.
  2. It is immediately offloaded back to the CPU.
  3. The reranker model is moved to the GPU to score the candidates.
  4. It is subsequently offloaded.
- **Cache Clearing:** `torch.cuda.empty_cache()` is called explicitly after operations to release VRAM.

This architecture allows the system to utilize large language models for generation without Out-Of-Memory (OOM) errors caused by holding embedding models in VRAM.

4 RAG - Generation Pipeline

The final stage of the system connects the retrieval engine with a Generative AI model to synthesize answers. Implemented in `rag.py`, the `RAGPipeline` class orchestrates the flow of data from user input to the final natural language response, leveraging the Google Gemini API for high-quality, low-latency generation.

4.1 Generative Model Integration

To achieve the reasoning capabilities required for a Subject Matter Expert (SME) agent, the system integrates Google's Gemini models (specifically configured for `gemini-2.5-flash-lite`) using the `langchain-google-genai` library.

- **API-Based Inference:** Unlike the embedding and reranking modules which run locally to preserve data privacy during indexing, the generation phase utilizes the `ChatGoogleGenerativeAI` class. This offloads the heavy computational load of text generation to the cloud, ensuring the system remains responsive even on consumer-grade hardware.
- **Configuration:** The generation parameters are tuned for factual accuracy rather than creative diversity:
  - `temperature: 0.7`: Balances fluency with determinism.
  - `top_p: 0.95`: Uses nucleus sampling to maintain output quality.
  - `max_output_tokens: 2048`: Allows for comprehensive, multi-paragraph explanations suitable for educational or technical content.

4.2 Prompt Engineering and Context Construction

A critical requirement of the project is to prevent "hallucinations"—situations where the AI invents information not present in the source text. We address this through strict Prompt Engineering using LangChain's message primitives.

4.2.1 Context Formatting

Before generation, the system aggregates the top N retrieved chunks (post-reranking) into a single context block. The chunks are delimited by visual separators to help the model distinguish between independent sources:

```
Context:
[Chunk Text 1]
---
[Chunk Text 2]
---
[Chunk Text 3]
```

4.2.2 System Prompts

We utilize a `SystemMessage` to enforce the agent's persona and behavioral constraints. The instruction set includes:

1. **Persona Definition:** "You are a helpful assistant for a K-12 student." (Configurable based on domain).
2. **Strict Grounding:** "You MUST answer the user's question only using the context provided. Do not use any outside knowledge."
3. **Negative Constraint:** Explicit instructions on how to handle missing information: "If the answer is not in the context, say: 'I'm sorry, but I couldn't find that information...'"

4.3 Pipeline Workflow

The `generate_answer` method executes the following synchronous workflow:

1. **Query Execution:** The user's natural language query is passed to the `Retriever` module.
2. **Resource Optimization:** Immediately after retrieval, the system invokes `retriever.offload_models_to_cpu()`. This clears the GPU VRAM used by the embedding and reranking models, preventing memory contention (OOM) and preparing the system state for the next request.
3. **Prompt Construction:** A structured prompt is assembled using `SystemMessage` (instructions) and `HumanMessage` (Context + Question).
4. **Invocation:** The prompt is sent to the Gemini API via `gen_model.invoke()`.
5. **Source Attribution:** The system extracts metadata (filenames) from the retrieved chunks and returns them alongside the answer, allowing users to verify the information source.

4.4 Error Handling

The pipeline includes robust exception handling for external API failures. If the Gemini API is unreachable or returns an error (e.g., quota limits), the system catches the exception via the `err_logger` and returns a graceful fallback message to the user, preventing a complete system crash.

5 Agentic Workflow and Orchestration

The core intelligence of the system is implemented in `agent_langgraph.py`, utilizing the **LangGraph** library to create a stateful, cyclic graph architecture. Unlike simple linear chains, this agent implements a dynamic task queue system capable of handling complex multi-intent user queries (e.g., "Create a report on Alkali Metals, generate a quiz, and email me the results").

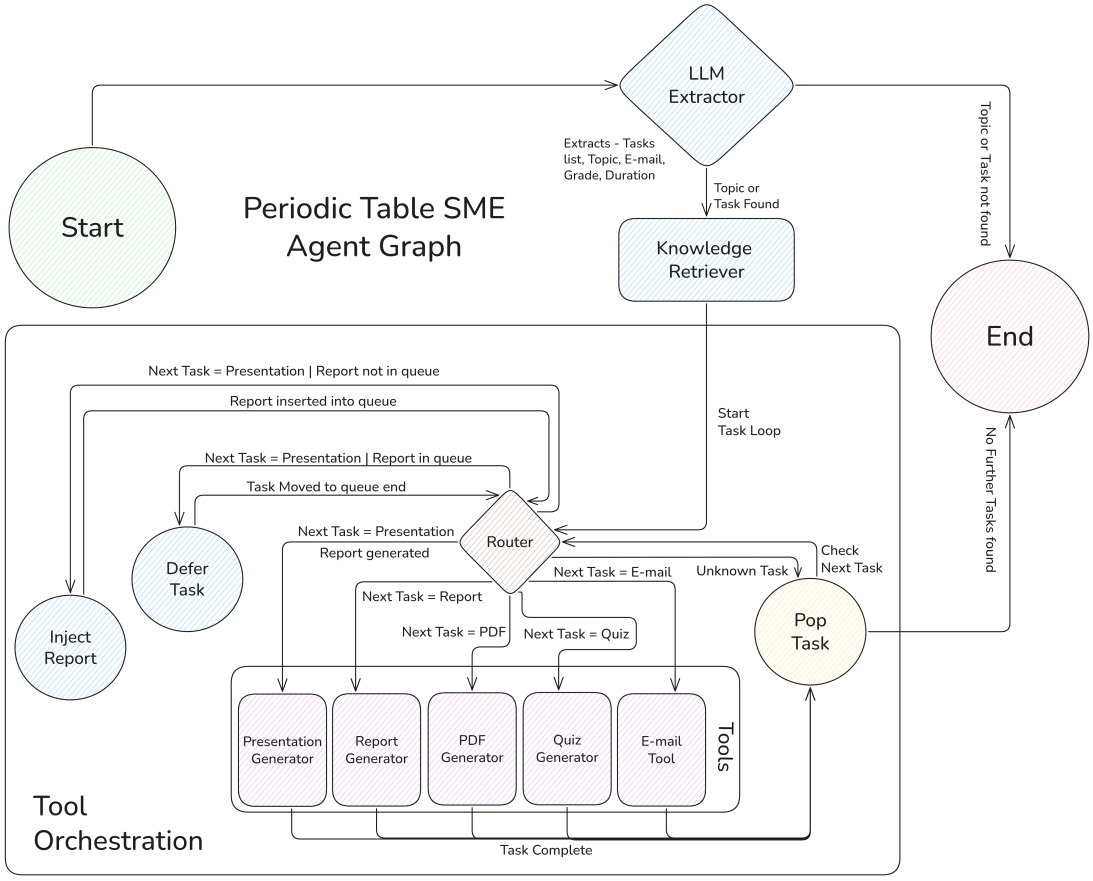

Figure 1: System architecture overview.

5.1 State Management

The agent maintains a persistent context through a typed dictionary, `AppState`, which evolves as the graph executes.

- **User Context:** Tracks `user_id`, `user_email`, and conversation history (`chat_summary`).
- **Task Queue:** A dynamic list (`tasks`) containing pending operations (e.g., `['report', 'quiz', 'presentation']`).
- **Content Accumulators:** Fields like `pdf_content` accumulate output from multiple steps to generate a single, comprehensive document at the end.
- **Artifact Paths:** Stores file paths for generated PDFs and PPTXs to be passed to the email node.

5.2 Intent Extraction and Planning

The entry point of the graph is the `llm_extractor` node. It employs a specialized prompt to parse the user's natural language request into a structured JSON plan.

- **Context-Awareness:** The extractor considers the `chat_summary` to resolve vague pronouns (e.g., "make a quiz on *that*").
- **Task Identification:** It populates the `tasks` list. For example, "Teach me about X and test me" maps to `['knowledge', 'quiz']`.
- **Email Resolution:** It intelligently extracts email addresses from the query or falls back to the registered user profile if available.

5.3 The Dynamic Task Loop

A key architectural innovation in this project is the **Cyclic Task Dispatcher**.

1. **Dispatcher (`route_tasks`):** This conditional edge inspects the head of the `tasks` queue.
2. **Execution:** It routes execution to the corresponding node (e.g., `generate_report`).
3. **State Update (`pop_task`):** After a node executes, the `pop_task` node removes the completed item from the queue.
4. **Recursion:** The graph cycles back to the dispatcher. If the queue is empty, it transitions to the document generation phase.

5.4 Self-Correction and Dependency Injection

The agent possesses reasoning capabilities to manage dependencies between tasks. A specific rule is implemented for presentations:

```
if next_task == "presentation" and not state.get("report_content"):
    return "inject_report_dependency"
```

- **Problem:** A user asks for a presentation without existing content.
- **Correction:** The agent detects the missing dependency (`report_content`).
- **Action:** It pauses the current task, injects a `report` task to the front of the queue, and re-queues the `presentation` task. This ensures the slide deck is always based on generated, factual content.

6 Tooling and Functional Capabilities

The functional capabilities of the SME are encapsulated in `multitools.py`. These tools are integrated via LangChain's `@tool` decorator and leverage the Google Gemini API ('gemini-2.5-flash') for content synthesis and various Python libraries for file manipulation.

6.1 Knowledge Retrieval Tool

The `knowledge_retrieval` tool acts as the bridge to the RAG pipeline.

- It initializes the `Retriever` class (from `retrieval.py`).
- **Resource Management:** Crucially, it implements an automatic cleanup step. After fetching context, it calls `retriever.offload_models_to_cpu()`. This ensures that the heavy embedding models do not occupy VRAM while the LLM is generating text, preventing Out-Of-Memory errors.

6.2 Content Generation Tools

- **Quiz Generator:** Accepts specific constraints (Grade, Duration) and uses the retrieved context to generate structured multiple-choice questions with an answer key.
- **Report Generator:** Synthesizes retrieval chunks into a coherent academic narrative with an introduction, body paragraphs, and a conclusion.

6.3 Document Generation Capabilities

6.3.1 Professional PDF Generation

The `pdf_generator` uses the `markdown_pdf` library.

- It aggregates content from all executed tasks (Knowledge + Report + Quiz).
- It renders Markdown formatting (headers, bold text) into a clean, paginated PDF document suitable for distribution.

6.3.2 Presentation Generation (PPTX)

The `presentation_generator` is a highly sophisticated tool built on `python-pptx`. It transforms raw text into a styled slide deck:

- **Content Parsing:** It parses Markdown headers (`##`) to delimit slides.
- **Junk Cleaning:** It employs regex to strip artifacts like "=====" dividers or leftover prompt instructions before processing.
- **Styling:** It applies a specific color theme (Dark Blue titles RGB(0, 51, 102)) and custom layouts.
- **Formatting:** It distinguishes between bullet points, numbered lists, and standard paragraphs, applying appropriate font sizes (e.g., 36pt for titles, 24pt for body) to ensure readability on projection screens.

6.4 Communication: Smart Email Tool

The `email_tool` uses `smtplib` to deliver results. It features a **Markdown-to-HTML Renderer** that converts the plain text output of the LLM into a visually appealing HTML email body:

- `## Headers` are converted to `<h2>` tags with corporate styling.
- `Quiz Options` are rendered into indented, monospace blocks for clarity.
- It automatically attaches the generated PDF and PPTX files if their paths exist in the state.

7 System Architecture and Implementation

The system is built upon a modern, decoupled architecture comprising a high-performance asynchronous backend, a persistent database layer with intelligent memory summarization, and a responsive frontend interface.

7.1 Backend API Server

The core application logic is hosted on a **FastAPI** server (`app.py`), chosen for its native support for asynchronous operations and automatic validation via Pydantic models.

7.1.1 Server Configuration

- **CORS Middleware:** The server is configured with `CORSMiddleware` to allow cross-origin requests from the frontend client (localhost:3000), enabling a clear separation of concerns between client and server.
- **Static File Serving:** A specialized endpoint mounts the `generated_documents` directory to `/files`, allowing the frontend to directly download generated PDFs and PPTXs via secure URLs.

7.1.2 Streaming Execution Endpoint

A critical feature of the architecture is the `/execute_stream` endpoint.

- **Server-Sent Events (SSE):** Instead of waiting for the entire agent workflow to complete, the server yields real-time updates using the `text/event-stream` media type.
- **Event Types:** The stream emits structured events such as:
  - `graph_update`: Updates from specific nodes in the LangGraph (e.g., "Generating Quiz...").
  - `ask_update`: Token-by-token streaming for standard RAG answers.
  - `end`: Signal for completion.
- **Persistence Integration:** Upon stream completion, the backend automatically commits the interaction history to the database and triggers a background summarization task, ensuring the user's context is saved without blocking the response.

7.2 Data Persistence and Memory Management

The system utilizes `SQLite` for robust local data storage, managed by the `DatabaseManager` class in `database.py`.

7.2.1 Database Schema

The schema is designed to support multi-user isolation and long-term memory:

1. **Users Table:** Stores credentials (hashed passwords) and profile metadata.
2. **Chat History:** Records individual message turns (`role`, `content`, `timestamp`) linked to specific `chat_ids`.
3. **Summaries:** Two distinct tables manage context:
   - `chat_summaries`: Scoped to a specific conversation, allowing the agent to recall context within a thread.
   - `user_summaries`: Global summaries (e.g., feedback analysis) that persist across sessions.

7.2.2 Intelligent Summarization Module

To handle token context limits, the system implements a summarization layer in `summarizer.py`.

- **Model:** Uses a lightweight LLM (`gemini-2.5-flash-lite`) to reduce cost and latency.
- **Process:** When a conversation concludes, the module retrieves the previous summary and the new messages. It uses a specialized prompt to synthesize them into a cohesive narrative (e.g., "The user initially asked about Alkali metals, then requested a quiz...").
- **Benefit:** This allows the main agent to understand the user's intent from a long conversation history without ingesting every single raw message, significantly optimizing input token usage.

7.3 Frontend Interface

The user interface is a single-page application (SPA) built with modern HTML5, CSS3 (utilizing CSS Variables for theming), and Vanilla JavaScript (`new_test.html`).

7.3.1 User Experience Features

- **Dual Modes:** Users can toggle between "Agent Mode" (for complex tasks like report generation) and "Ask Mode" (for quick Q&A).
- **Real-Time Feedback:** The interface parses the SSE stream to display a "typing" effect for answers and updates a status bar with the agent's current internal step (e.g., "Creating presentation slides...").
- **Rich Text Rendering:** Incoming markdown from the agent is parsed client-side to render headers, lists, and bold text, while sanitizing user input to prevent XSS attacks.

7.3.2 Session Management

- **Sidebar Navigation:** A dynamic sidebar lists past chat sessions, fetched from the backend, allowing users to switch contexts instantly.
- **Theme Engine:** A robust theming system supports Light and Dark modes, persisting user preference in `localStorage`.

8 Experiments and Results

We conducted a comprehensive evaluation of the Subject Matter Expert (SME) Agent to assess its capabilities in both quantitative accuracy and qualitative reasoning. The experiments compared the performance of three distinct Large Language Models (LLMs) within our RAG pipeline on a curated test set of chemistry questions ($N = 135$).

8.1 Quantitative Analysis

The primary metric for success was **Accuracy**, defined as the percentage of instances where the agent's predicted option (A, B, C, or D) matched the ground truth. To provide a holistic view of performance, we also calculated Weighted Precision, Recall, and F1-Score.

The results, summarized in Table 1, demonstrate that while commercial models still lead in performance, quantized local models have reached a viable threshold for deployment.

Model	Accuracy	Precision	Recall	F1-Score
Gemini 2.5 Flash	**0.811**	**0.905**	**0.811**	**0.824**
Gemini 2.5 Flash Lite	0.730	0.850	0.730	0.753
Qwen 1.7B (8-bit)	0.642	0.701	0.617	0.653

Table 1: Performance comparison of LLMs on the Chemistry QA Task.

Gemini 2.5 Flash achieved the highest performance across all metrics (81.1%), demonstrating superior reasoning capabilities. **Gemini 2.5 Flash Lite** followed with 73.0%, showing a slight degradation in complex reasoning. Notably, the updated **Qwen 1.7B (8-bit)** model achieved a respectable 64.2% accuracy, proving that even small, quantized models can effectively leverage retrieved context for standard domain questions.

8.2 Qualitative Analysis

A deeper inspection of the model outputs reveals distinct behavioral patterns that explain the quantitative variance.

8.2.1 Reasoning Capabilities: Flash vs. Lite

The standard Flash model demonstrated robust fact-checking capabilities. For instance, in **Question 29**, which asked to identify the *incorrect* statement about D.I. Mendeleev, **Gemini 2.5 Flash** correctly identified option (b) ("At the time, he proposed Periodic Table of elements structure of atom was known") by cross-referencing historical scientific context. In contrast, **Gemini 2.5 Flash Lite** failed to detect the anachronism and incorrectly flagged option (a) as the answer, showcasing a tendency to hallucinate factual errors when distinguishing between plausible distractors.

8.2.2 Consistency and Instruction Following

We observed cases where the "Lite" model struggled with output consistency. In **Question 28** (identifying coloured and paramagnetic ions), the Lite model's internal reasoning chain correctly identified the chemical properties and stated "The correct option is (a)" in its text generation. However, its final structured prediction was option (b). This mismatch indicates that while the smaller model can retrieve correct information, it occasionally fails to maintain consistency between its reasoning chain and its final classification token.

8.2.3 Viability of Local Models

The **Qwen 1.7B (8-bit)** model showed a marked improvement over previous baselines. It successfully answered factual queries such as **Question 1** (IUPAC naming of platinum complexes), correctly predicting option (a). This suggests that for direct fact-retrieval tasks where the answer is explicitly present in the context chunks, quantized local models offer a cost-effective alternative to commercial APIs, though they still lag in multi-step deductive reasoning.

9 Bonus Implementations

In alignment with the advanced requirements of the Major Project, we successfully implemented four distinct bonus features. These implementations enhance the system's robustness, retrieval accuracy, and adaptability, transitioning it from a basic prototype to a production-grade Subject Matter Expert (SME) agent.

9.1 Automated Batch Ingestion Pipeline

Implementation Detail: We implemented a parallelized ingestion system in `ingestion.py` utilizing Python's `multiprocessing` library.

- **Parallel Execution:** The system detects available CPU cores and spawns a worker pool (configurable via `--batch-size`). The `ingest_directory` function uses `pool.starmap` to distribute file processing tasks concurrently, significantly reducing the I/O bottleneck inherent in file reading.
- **Robust Logging:** A dual-channel logging system was architected. Successful extractions are recorded in `ingest.log`, while failures (e.g., corrupted PDFs, encoding errors) are captured in `errors.log` with full stack traces. This ensures that a single bad file does not halt the entire pipeline.

9.2 Relevance Reranking Mechanism

Implementation Detail: To address the limitations of pure vector similarity (which often retrieves semantically similar but factually irrelevant chunks), we integrated a Cross-Encoder model in `retrieval.py`.

- **Two-Stage Retrieval:** The system first retrieves the top-K (e.g., 20) candidates using fast cosine similarity from Elasticsearch.
- **Semantic Reranking:** These candidates are passed to the `BAAI/bge-reranker-base` model. Unlike the bi-encoder used for indexing, this model processes the query and document pair simultaneously, outputting a precise relevance score.
- **Resource Management:** The reranker is loaded dynamically onto the GPU only during the inference window and immediately offloaded to the CPU via `unload_reranker()`, preventing VRAM saturation.

9.3 Self-Learning and Adaptation via Feedback Loops

Implementation Detail: We constructed a persistent feedback loop connecting the frontend, database, and agent orchestration layers.

- **Capture:** The `/add_feedback` endpoint in `app.py` captures user sentiment and specific critiques.
- **Synthesis:** The `summarizer.py` module uses a lightweight LLM (`gemini-2.5-flash-lite`) to aggregate raw feedback into a concise "Feedback Summary" (e.g., "Users frequently request harder quiz questions").
- **Adaptation:** This summary is injected into the `AppState` of the LangGraph agent. The `llm_extractor` prompt explicitly instructs the model: "You must consider the user's previous history and feedback." This allows the agent to adjust its task planning and content generation style based on cumulative user input.

9.4 OCR Fallback for Scanned Documents

Implementation Detail: Recognizing that real-world datasets often contain scanned images wrapped in PDF containers, we implemented an OCR fallback in `ingestion.py`.

- **Heuristic Detection:** The system analyzes the text extracted by standard tools. If the character count is below a threshold (20 chars) for a given page, it flags the page as "scanned."
- **Visual Extraction:** It triggers `pytesseract` (Tesseract OCR) to render the page as an image and extract text from the visual layer, ensuring no data is lost during ingestion.

9.5 Input Guardrails and Sanitization

Implementation Detail: We implemented security measures at both the client and server levels to prevent injection attacks and ensure safe rendering.

- **Client-Side Sanitization:** The frontend implements strict character escaping (`.replace(/</g, '<')`) for user inputs before rendering them in the chat DOM, neutralizing Cross-Site Scripting (XSS) vectors.
- **Structured Injection Defense:** By utilizing a strict JSON schema in the `llm_extractor` node (`agent_langgraph.py`), raw user queries are compartmentalized into typed fields (Task, Topic, Grade). This prevents "jailbreak" style prompts from overriding the agent's core logic flow.